

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_58a19e3e-3ce\agent-aa4b10e93e775c753.jsonl`  
- SHA-256: `a787ad1758cbfc1c43f2ee66fa543e95bbe6095750760a45e906f6aac75cf24d`  
- Source modified: `2026-07-06T23:22:36+00:00`  
- Imported at: `2026-07-08T16:00:06+00:00`  
- project: `wf_58a19e3e-3ce`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T23:21:09.996Z)

Reviewing GitHub PR #54 "feat(policy): add F5 deep inspection budget boundary" (branch feat/f5-deep-inspection-budget -> main). ADR 0012 F5.
+364/-4, 3 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr54 (read there or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f5-deep-inspection-budget:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr54/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F5 DOES (policy.py) ? BOUNDARY ONLY, no downloader/ffprobe/hash/scan (explicitly deferred):

```
- PolicyBudget(frozen dataclass): max_bytes_per_item, timeout_s, max_concurrent, + private  
_semaphore built in __post_init__ (via object.__setattr__)  
  when max_concurrent>0. async inspect(fetcher, content_item): 'async with self._semaphore:  
return await asyncio.wait_for(fetcher.inspect(item,self), timeout=timeout_s)'.  
- ContentFetcher(Protocol): async inspect(content_item, budget)->StageResult.  
PolicyContext.fetcher/budget now typed (were object|None in F1).  
- load_deep_inspection_enabled(policy): deep_inspect bool OR {enabled: bool}; invalid ->  
ValueError -> stage fail-OPEN (pass, invalid_deep_inspection_config).  
- DeepInspectionStage (Tier 4, LAST after topic/url/caption): opt-out if not enabled -> pass.  
_deep_inspection_budget_result checks (in order, all  
  BEFORE calling fetcher): budget present, budget valid (max_bytes/timeout/max_concurrent all  
>0), content_item.size_bytes present, size<=max_bytes ->  
  returns a FAIL StageResult on any violation. Then if fetcher missing -> fail. Else  
budget.inspect(...) under semaphore+wait_for; TimeoutError -> fail 'budget exceeded'.  
- Live wiring (_evaluate_policy_stages) creates PolicyContext WITHOUT fetcher/budget (both  
None), so deep_inspect enabled -> stage fails 'budget missing'  
  -> video DROPPED (fail-closed). This is intentional/dormant (no /set deep_inspect command  
exists).
```

VERIFIED by main reviewer: gate green (543 passed @ 94.57%); budget checks run BEFORE the
fetcher; deep_inspect enabled-without-budget drops the item
(fail-closed, safe direction); default (deep_inspect unset) unaffected. Comprehensive tests
(missing/invalid budget, missing size, size-exceeded, timeout,
concurrency max_active==1, missing fetcher, valid delegate).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No
praise. Concrete failure scenario, ranked.
Severity: high (budget/timeout/concurrency BYPASS ? heavy work runs unbounded, or a real bug),
medium (real latent/design), low (note/hygiene/intentional-fail-closed).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read cited code, run a repro with
PYTHONPATH=C:/Users/avidu/Projects/_wt-pr54/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong
behaviour. REFUTED if handled / can't occur /

intentional-and-safe / out-of-scope-deferred. PLAUSIBLE if unsettled. Corrected severity (a real budget bypass = high; intentional fail-closed note = low).

FINDING (config_wiring): Real TimeoutError from fetcher internals is misclassified as a budget-timeout event

severity medium | src/tg_video_filter/policy.py:709

summary: DeepInspectionStage.evaluate wraps budget.inspect in `except TimeoutError`, so any genuine TimeoutError raised inside the fetcher (unrelated to the budget) is reported as budget_exceeded/exceeded_budget=timeout_s with the budget's timeout value, not the actual failure.

scenario: With budget timeout_s=10.0 and a fetcher that raises TimeoutError immediately (e.g. an inner network/socket timeout that completes in milliseconds), the stage returns FAIL reason='deep inspection budget exceeded', metadata={'budget_exceeded': True, 'exceeded_budget': 'timeout_s', 'timeout_s': 10.0}. Operators/telemetry see a fabricated 10s budget overrun for an error that was not a budget overrun. Verified by repro: fetcher raising TimeoutError well within a 10s budget produced exactly that mislabeled StageResult.

evidence: policy.py evaluate(): `try: return await ctx.budget.inspect(ctx.fetcher, ctx.content\_item) except TimeoutError: return StageResult.fail(... budget\_exceeded=True, exceeded\_budget='timeout\_s', timeout\_s=ctx.budget.timeout\_s)`. asyncio.wait_for raises asyncio.TimeoutError (== builtin TimeoutError in 3.11+) on real budget expiry, but a builtin TimeoutError from fetcher.inspect propagates through wait_for unchanged and is caught by the same handler, indistinguishable from a true budget timeout. Repro output: 'passed= False | reason= deep inspection budget exceeded | meta= {budget_exceeded: True, exceeded_budget: timeout_s, timeout_s: 10.0}' for a fetcher that raised TimeoutError instantly.

rec: Distinguish the two: let PolicyBudget.inspect catch the wait_for timeout itself and re-raise a dedicated sentinel (e.g. a BudgetExceeded exception), or check elapsed time, so the stage only reports budget_exceeded when the budget actually expired. A fetcher-internal TimeoutError should surface as a distinct failure reason. Direction stays fail-closed either way, so this is correctness/observability, not a bypass.

2. assistant (2026-07-06T23:21:11.777Z)

I'll verify this finding by examining the code and running a repro.

3. user (2026-07-06T23:21:12.901Z)

```
660         if caption_truncated:
661             metadata["caption_truncated"] = True
662             metadata["max_caption_scan_chars"] = _MAX_CAPTION_SCAN_CHARS
663         if config.invalid_patterns:
664             metadata["ignored_invalid_caption_patterns"] = list(config.invalid_patterns)
665         if config.ignored_term_count:
666             metadata["ignored_caption_term_count"] = config.ignored_term_count
667         if config.ignored_pattern_count:
668             metadata["ignored_caption_pattern_count"] = config.ignored_pattern_count
669         if timed_out_patterns:
670             metadata["timed_out_caption_patterns"] = list(timed_out_patterns)
671         return metadata
672
673
674     class DeepInspectionStage:
675         """ADR 0012 Tier 4 opt-in boundary for budgeted heavy inspection."""
676
677         stage_name = "deep-inspection"
678
679         async def evaluate(self, ctx: PolicyContext) -> StageResult:
680             try:
681                 enabled = load_deep_inspection_enabled(ctx.policy)
682             except (json.JSONDecodeError, ValueError) as exc:
683                 log.warning(
684                     "deep inspection disabled due to invalid config policy_id=%s",
685                     ctx.policy.id,
686                     exc_info=True,
687                 )
688             return StageResult.pass_(
689                 self.stage_name,
```

```

690         f"invalid deep inspection config ignored: {exc}",
691         invalid_deep_inspection_config=True,
692     )
693     if not enabled:
694         return StageResult.pass_(self.stage_name, "deep inspection disabled")
695
696     budget_result = _deep_inspection_budget_result(ctx)
697     if budget_result is not None:
698         return budget_result
699
700     if ctx.fetcher is None:
701         return StageResult.fail(
702             self.stage_name,
703             "deep inspection fetcher missing",
704             missing_fetcher=True,
705         )
706
707     assert ctx.budget is not None
708     try:
709         return await ctx.budget.inspect(ctx.fetcher, ctx.content_item)
710     except TimeoutError:
711         return StageResult.fail(
712             self.stage_name,
713             "deep inspection budget exceeded",
714             budget_exceeded=True,
715             exceeded_budget="timeout_s",
716             timeout_s=ctx.budget.timeout_s,
717         )
718
719
720 def _deep_inspection_budget_result(ctx: PolicyContext) -> StageResult | None:
721     budget = ctx.budget
722     if budget is None:
723         return StageResult.fail(
724             DeepInspectionStage.stage_name,
725             "deep inspection budget missing",
726             missing_budget=True,
727         )
728     if budget.max_bytes_per_item <= 0 or budget.timeout_s <= 0 or budget.max_concurrent
729     <= 0:
730         return StageResult.fail(
731             DeepInspectionStage.stage_name,
732             "deep inspection budget invalid",
733             invalid_budget=True,
734             max_bytes_per_item=budget.max_bytes_per_item,
735             timeout_s=budget.timeout_s,
736             max_concurrent=budget.max_concurrent,
737         )
738     size_bytes = ctx.content_item.size_bytes
739     if size_bytes is None:
740         return StageResult.fail(
741             DeepInspectionStage.stage_name,
742             "deep inspection budget cannot be enforced without content size",
743             missing_size_bytes=True,
744             max_bytes_per_item=budget.max_bytes_per_item,
745         )
746     if size_bytes > budget.max_bytes_per_item:
747         return StageResult.fail(
748             DeepInspectionStage.stage_name,
749             "deep inspection budget exceeded",
750             budget_exceeded=True,

```

4. assistant (2026-07-06T23:21:18.917Z)

The finding is about the `except TimeoutError` at line 710. Let me look at the

`PolicyBudget.inspect` method to understand how the timeout is raised.

5. user (2026-07-06T23:21:20.613Z)

```
44: class PolicyBudget:
45:     _semaphore: asyncio.Semaphore | None = field(
46:         def __post_init__(self) -> None:
47:             object.__setattr__(self, "_semaphore", asyncio.Semaphore(self.max_concurrent))
48:     async def inspect(self, fetcher: ContentFetcher, content_item: ContentItem) ->
49: StageResult:
50:         if self._semaphore is None:
51:             async with self._semaphore:
52:                 return await asyncio.wait_for(
53:         async def inspect(self, content_item: ContentItem, budget: PolicyBudget) -> StageResult:
```

6. user (2026-07-06T23:21:23.582Z)

```
44     class PolicyBudget:
45         """Bounded execution budget for opt-in deep inspection."""
46
47         max_bytes_per_item: int
48         timeout_s: float
49         max_concurrent: int
50         _semaphore: asyncio.Semaphore | None = field(
51             default=None,
52             init=False,
53             repr=False,
54             compare=False,
55         )
56
57         def __post_init__(self) -> None:
58             if self.max_concurrent > 0:
59                 object.__setattr__(self, "_semaphore",
60 asyncio.Semaphore(self.max_concurrent))
61
62         async def inspect(self, fetcher: ContentFetcher, content_item: ContentItem) ->
63 StageResult:
64             """Run ``fetcher`` under the timeout and concurrency budget."""
65             if self._semaphore is None:
66                 raise ValueError("max_concurrent must be positive")
67             async with self._semaphore:
68                 return await asyncio.wait_for(
69                     fetcher.inspect(content_item, self),
70                     timeout=self.timeout_s,
71                 )
72
73     class ContentFetcher(Protocol):
74         """Fetcher used only by opt-in Tier 4 policy stages."""
75
76         async def inspect(self, content_item: ContentItem, budget: PolicyBudget) ->
77 StageResult:
78             """Inspect content under ``budget`` and return a policy result."""
79             ...
```

7. assistant (2026-07-06T23:21:27.108Z)

The finding's core claim is confirmed structurally: `asyncio.wait\_for` raises `asyncio.TimeoutError`, which in Python 3.11+ is an alias for the builtin `TimeoutError`. A fetcher that raises a plain `TimeoutError` internally propagates through `wait\_for` unchanged and is caught by the same `except TimeoutError` handler at line 710. Let me verify the Python version and run a repro.

8. user (2026-07-06T23:21:28.296Z)

Python 3.13.13

9. assistant (2026-07-06T23:21:31.662Z)

Python 3.13, so `asyncio.TimeoutError is TimeoutError` is True. Now let me check the ContentItem / StageResult / PolicyContext structures needed for the repro.

10. user (2026-07-06T23:21:33.445Z)

```
81:class PolicyContext:
92:class StageResult:
101:    def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
110:    def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
737:    size_bytes = ctx.content_item.size_bytes
738:    if size_bytes is None:
742:        missing_size_bytes=True,
745:    if size_bytes > budget.max_bytes_per_item:
751:        size_bytes=size_bytes,
```

11. user (2026-07-06T23:21:35.914Z)

```
80    @dataclass(frozen=True)
81    class PolicyContext:
82        """Immutable snapshot available to a policy stage."""
83
84        content_item: ContentItem
85        source: Source
86        policy: Policy
87        fetcher: ContentFetcher | None = None
88        budget: PolicyBudget | None = None
89
90
91    @dataclass(frozen=True)
92    class StageResult:
93        """Outcome of one policy stage."""
94
95        passed: bool
96        reason: str
97        stage_name: str
98        metadata: dict[str, object] = field(default_factory=dict)
99
100    @staticmethod
101    def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
102        return StageResult(
103            passed=True,
104            reason=reason,
105            stage_name=stage_name,
106            metadata=metadata,
107        )
108
109    @staticmethod
110    def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
111        return StageResult(
112            passed=False,
113            reason=reason,
114            stage_name=stage_name,
115            metadata=metadata,
116        )
117
118
119    class PolicyStage(Protocol):
```

12. assistant (2026-07-06T23:21:38.403Z)

Now let me find the ContentItem, Source, Policy constructors to build a minimal repro.

13. user (2026-07-06T23:21:39.919Z)

```
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\db.py:1317:    as ``topics`` /
``security_checks`` / ``deep_inspect`` instead of validating
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\models.py:123:class Source(BaseModel):
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\models.py:149:class ContentItem(BaseModel):
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\models.py:186:class Policy(BaseModel):
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\policy.py:294:def
load_deep_inspection_enabled(policy: Policy) -> bool:
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\policy.py:299:    raw_deep_inspect =
decoded.get("deep_inspect", False)
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\policy.py:300:    if
isinstance(raw_deep_inspect, bool):
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\policy.py:301:        return
raw_deep_inspect
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\policy.py:302:    if
isinstance(raw_deep_inspect, dict):
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\policy.py:303:        enabled =
raw_deep_inspect.get("enabled", False)
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\policy.py:306:    raise
ValueError("deep_inspect must be a boolean or an object with boolean enabled")
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\policy.py:681:        enabled =
load_deep_inspection_enabled(ctx.policy)
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\policy.py:691:
invalid_deep_inspection_config=True,
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\policy.py:696:        budget_result =
_deep_inspection_budget_result(ctx)
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\policy.py:720:def
_deep_inspection_budget_result(ctx: PolicyContext) -> StageResult | None:
C:\Users\avidu\Projects\_wt-pr54\src\tg_video_filter\policy.py:788:
"load_deep_inspection_enabled",
```

14. assistant (2026-07-06T23:21:42.911Z)

I have enough to build a repro. Rather than fully constructing pydantic models, I'll test the exact code path in question directly against `PolicyBudget.inspect` and the `evaluate` handler logic. Let me build the repro using the real classes.

15. user (2026-07-06T23:21:44.055Z)

```
149     class ContentItem(BaseModel):
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153         (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154         NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155         ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156         wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157         """
158
159         model_config = ConfigDict(frozen=True)
160
161         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162         account_id: int = Field(description="FK ? accounts.id")
163         source_id: int = Field(description="FK ? sources.id")
164         provider: str = Field(default="telegram", description="Provider namespace")
165         provider_message_id: str = Field(
166             description="Provider-native message id (Telegram message_id, ?)"
167         )
168         content_type: str = Field(default="video", description="'video' | 'link' |
```

```

'article')
169         text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
170         link_url: str | None = Field(default=None, description="For link/article posts
(Phase 1: NULL)")
171         media_ref: str | None = Field(default=None, description="Provider media reference")
172         document_id: str | None = Field(
173             default=None,
174             description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175         )
176         duration_s: int | None = None
177         width: int | None = None
178         height: int | None = None
179         size_bytes: int | None = None
180         mime_type: str | None = None
181         dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
per account")
182         matched: bool = False
183         seen_at: datetime = Field(default_factory=utcnow)
184
185
186     class Policy(BaseModel):
187         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189         Generalizes the legacy per-account ``filter_config`` row so one account can
190         own multiple policies (enables ADR 0009's per-route filter overrides) and so
191         future enrichment/security stages grow into the same ``config_json`` column.
192         Exactly one policy per account carries ``is_default=True``; the legacy
193         ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
194         """
195
196         model_config = ConfigDict(frozen=True)
197
198         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")

```

16. assistant (2026-07-06T23:21:47.616Z)

I'll write the repro exercising the real `DeepInspectionStage.evaluate` with a fetcher that raises a plain `TimeoutError` instantly, well within a 10s budget.

17. assistant (2026-07-06T23:21:55.744Z)

```

import asyncio
import time
from tg_video_filter import policy as P
from tg_video_filter.models import ContentItem, Source, Policy

```

Confirm alias identity

```
print("asyncio.TimeoutError is TimeoutError:", asyncio.TimeoutError is TimeoutError)
```

```

class InstantTimeoutFetcher:
    async def inspect(self, content_item, budget):
        # Simulate an inner socket/network timeout that fires immediately,
        # unrelated to the budget's timeout_s.
        raise TimeoutError("inner socket timeout")

def make_ctx():
    item = ContentItem(
        account_id=1, source_id=1, provider_message_id="m1",
        size_bytes=100, dedup_key="k1",
    )
    src = Source(id=1, account_id=1, kind="telegram", identifier="chan")
    pol = Policy(id=1, account_id=1, name="p", config_json={"deep_inspect": True})
    budget = P.PolicyBudget(max_bytes_per_item=1_000_000, timeout_s=10.0, max_concurrent=1)

```

```

        return P.PolicyContext(content_item=item, source=src, policy=pol,
                               fetcher=InstantTimeoutFetcher(), budget=budget)

async def main():
    ctx = make_ctx()
    stage = P.DeepInspectionStage()
    t0 = time.monotonic()
    res = await stage.evaluate(ctx)
    elapsed = time.monotonic() - t0
    print(f"elapsed={elapsed:.4f}s (budget timeout_s=10.0)")
    print("passed=", res.passed, "| reason=", res.reason, "| meta=", res.metadata)

asyncio.run(main())

```

18. user (2026-07-06T23:21:55.770Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro_pr54.py (file state is current in your context ? no need to Read it back)

19. assistant (2026-07-06T23:21:57.879Z)

I need to check the Source and Policy field names. Let me check the constructors quickly.

20. user (2026-07-06T23:21:59.018Z)

```

123     class Source(BaseModel):
124         """A configured place we ingest content from (ADR 0011).
125
126         Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127         providers add their own source kinds). ``provider_source_id`` is the
128         provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129         """
130
131         model_config = ConfigDict(frozen=True)
132
133         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134         account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135         provider: str = Field(default="telegram", description="Provider namespace")
136         provider_source_id: str = Field(
137             description="Provider-native source id (Telegram channel_id, ?)"
138         )
139         title: str | None = None
140         kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141         watched: bool = Field(
142             default=False,
143             description="ADR 0009 selective scanning: ingest from this source iff True",
144         )
145         seen_first_at: datetime = Field(default_factory=_utcnow)
146         last_seen_at: datetime | None = None
147
148

```

21. user (2026-07-06T23:22:01.312Z)

```

198         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
199         account_id: int = Field(description="FK ? accounts.id")
200         name: str = Field(description="Policy name; 'default' for the account default
policy")
201         config_json: str = Field(
202             description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
203         )
204         is_default: bool = Field(
205             default=False, description="Exactly one per account (UNIQUE partial index)"

```



```

206         )
207         created_at: datetime = Field(default_factory=_utcnow)
208         updated_at: datetime = Field(default_factory=_utcnow)
209
210
211     # --- Track C: destinations, routing, delivery state (ADR 0011) -----
212     #
213     # These tables were created empty by Track B (_SCHEMA); Track C populates
214     # destinations (backfilled from filter_config_deprecated.config_json), writes
215     # delivery_attempts (replacing the legacy videos.forwarded boolean), and
216     # defines the models for per-destination delivery state. The source/destination
217     # invariant (a channel is either source or destination, never both) is enforced

```

22. assistant (2026-07-06T23:22:03.230Z)

I need to fix Source constructor (uses `provider\_source\_id`, not `identifier`).

23. user (2026-07-06T23:22:06.253Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro_pr54.py has been updated successfully. (file state is current in your context ? no need to Read it back)

24. user (2026-07-06T23:22:10.855Z)

```

asyncio.TimeoutError is TimeoutError: True
elapsed=0.0001s (budget timeout_s=10.0)
passed= False | reason= deep inspection budget exceeded | meta= {'budget_exceeded': True,
'exceeded_budget': 'timeout_s', 'timeout_s': 10.0}

```

25. assistant (2026-07-06T23:22:25.678Z)

The repro confirms the finding exactly. A fetcher raising a plain `TimeoutError` in 0.0001s (well within a 10s budget) yields a StageResult labeled `reason='deep inspection budget exceeded'`, `metadata={budget\_exceeded: True, exceeded\_budget: 'timeout\_s', timeout\_s: 10.0}`. Telemetry/operators see a fabricated 10-second budget overrun for an error that was not a budget overrun.

Now the adversarial part ? can I refute it? Let me consider counterarguments:

1. ****Is `except TimeoutError` actually new in this PR's diff?**** Yes ? the entire `DeepInspectionStage` class (lines 674-717), including the `except TimeoutError` handler, is new in this PR. Confirmed at policy.py:710.
2. ****Does `wait\_for` really let an inner builtin `TimeoutError` propagate unchanged?**** In Python 3.11+, `asyncio.wait\_for` wraps its own cancellation-driven timeout as `TimeoutError`. But if the coroutine *itself* raises before the timeout fires, that exception propagates as-is. Since `asyncio.TimeoutError is TimeoutError`, the handler cannot distinguish them. Repro confirms this empirically.
3. ****Is this a genuine wrong behavior or intentional/safe?**** The direction stays fail-closed (item still dropped), so it's not a budget *bypass* ? no heavy work runs unbounded. The defect is purely correctness/observability: misclassification of the failure reason and fabricated metadata. This matches the finding author's own scope note ("correctness/observability, not a bypass").
4. ****Could a real fetcher plausibly raise builtin `TimeoutError`?**** Yes ? `socket.timeout` is an alias of `TimeoutError` since Python 3.10, and `asyncio` stream operations, subprocess timeouts, and many network libraries raise `TimeoutError`. This is a realistic scenario for a fetcher doing network I/O (which is exactly what a deep-inspection fetcher would do), even though the fetcher is currently deferred/unwired.

The finding is technically CONFIRMED. On severity: there is no bypass ? heavy work is not run unbounded, and the fail-closed direction holds. The impact is mislabeled telemetry/reason for a

failure mode that only arises once a real fetcher is wired (currently deferred). This is a latent correctness/observability defect. The finding's assigned severity of ****medium**** is appropriate ? it's a real defect in shipped diff code, not merely hygiene, but it's not a security/budget bypass and only manifests with a future wired fetcher. I'll keep medium.

26. user (2026-07-06T23:22:36.885Z)

Structured output provided successfully